Subject: Machine Learning1 **Level:** GLSI2	PRISTINI SCHOOL OF AI	**Academic year:** 2025/2026 **A.Prof**: Ameni YOUSSFI

Lab n ° 2

Machine learning Linear Regression

Part 2: Exploratory Data Analysis (EDA).

1. Visualize the relationship between the target variable and the independent variables using regplot.
2. Examine the correlation between variables using function corr(), Provide a summary of your observations.

Part 3: Building the Linear Regression Model.

1. Split the data into training (70%) and testing (30%) sets.
2. Train a multiple linear regression model using at these four independent variables
3. Display the coefficients and intercept of your model. Interpret what these values mean in the context of the dataset.

Part 4 : Model Evaluation

1. Evaluate your model using metrics such as Mean Squared Error (MSE), and R-squared (R^2).
2. Provide an interpretation of your model's performance. How well does it predict the target variable?

Dataset Overview

The dataset used in this lab is house_price_regression_dataset.csv. It contains 1,000 records with 7 independent variables and 1 target variable.

Column Name	Type	Description
Square_Footage	Integer	Total area of the house in square feet
Num_Bedrooms	Integer	Number of bedrooms
Num_Bathrooms	Integer	Number of bathrooms
Year_Built	Integer	Year the house was constructed
Lot_Size	Float	Size of the lot in acres
Garage_Size	Integer	Number of cars the garage can hold
Neighborhood_Quality	Integer	Quality score of the neighborhood (1–10)
House_Price	Float	Target: Sale price of the house (in USD)

Part 2 — Exploratory Data Analysis (EDA)

Learning Objectives
• Visualize relationships between the target variable (House_Price) and each predictor using regplot.
• Measure pairwise correlations using the corr() function.
• Identify the most influential predictors before building any model.

2.1 Visualizing Relationships (regplot)

A regression plot (regplot) combines a scatter plot with a fitted regression line. It helps you visually assess whether a linear relationship exists between two variables.

Step 1 : Import libraries

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv('house_price_regression_dataset.csv')
```

Step 2: Plot House_Price vs each independent variable

```
features = ['Square_Footage', 'Num_Bedrooms', 'Num_Bathrooms',
            'Year_Built', 'Lot_Size', 'Garage_Size', 'Neighborhood_Quality']

fig, axes = plt.subplots(nrows=3, ncols=3, figsize=(15, 12))
axes = axes.flatten()

for i, feature in enumerate(features):
    sns.regplot(x=df[feature], y=df['House_Price'], ax=axes[i],
                scatter_kws={'alpha': 0.4, 'color': '#2E5FA3'},
                line_kws={'color': '#E74C3C', 'linewidth': 2})
    axes[i].set_title(f'House_Price vs {feature}', fontsize=11, fontweight='bold')
    axes[i].set_xlabel(feature)
    axes[i].set_ylabel('House Price (USD)')

# Hide the empty subplot
axes[-1].set_visible(False)
plt.suptitle('Regression Plots — House Price vs Predictors', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()
```

Task 2.1 Your Observations
After running the code above, answer the following questions:
1. Which variable appears to have the strongest positive linear relationship with House_Price?
……
2. Are there any variables that show little or no linear trend?
→ ……..
3. Do you notice any outliers or unusual data points in any plot?
→ ……

2.2 Correlation Matrix , corr()

The Pearson correlation coefficient measures the linear strength between two numeric variables. Values close to +1 indicate a strong positive relationship, values close to −1 a strong negative one, and values near 0 suggest no linear relationship.

```
# Compute correlation matrix
corr_matrix = df.corr()
print(corr_matrix)

# Visualize as a heatmap
plt.figure(figsize=(10, 7))
sns.heatmap(corr_matrix,
            annot=True,
            fmt='.2f',
            cmap='coolwarm',
            center=0,
            linewidths=0.5,
            square=True)
plt.title('Correlation Matrix — House Price Dataset', fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()

# Correlation with target only
print(corr_matrix['House_Price'].sort_values(ascending=False))
```

Part 3 — Building the Linear Regression Models

Learning Objectives
• Split data correctly into training and testing sets.
• Train simple, multiple, and polynomial regression models.
• Interpret model coefficients and intercepts in a real-world context.

3.1 Train / Test Split (70 % / 30 %)

```
from sklearn.model_selection import train_test_split

# Features used in all three models
X = df[['Square_Footage', 'Num_Bedrooms', 'Num_Bathrooms', 'Neighborhood_Quality']]
y = df['House_Price']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=42)

print('Training samples :', X_train.shape[0])
print('Testing  samples :', X_test.shape[0])
```

3.2 Simple Linear Regression

A simple linear regression uses ONE independent variable to predict the target. We will use Square_Footage as it showed the highest correlation with House_Price.

```
from sklearn.linear_model import LinearRegression

# Use only one feature
X_simple_train = X_train[['Square_Footage']]
X_simple_test  = X_test[['Square_Footage']]

model_simple = LinearRegression()
model_simple.fit(X_simple_train, y_train)

print('=== Simple Linear Regression ===')
print(f'Intercept  (β0): {model_simple.intercept_:,.2f}')
print(f'Coefficient (β1) for Square_Footage: {model_simple.coef_[0]:,.2f}')
print()
```

```
print('Equation: House_Price = β0 + β1 × Square_Footage')
print(f'          House_Price = {model_simple.intercept_:,.0f} + {model_simple.coef_[0]:,.2f}
× Square_Footage')
```

Task 3.2 — Interpret Simple Regression
1. What does the intercept β0 represent in this context?
→ __.....
2. What does the coefficient β1 for Square_Footage mean?
(Hint: if Square_Footage increases by 1 sq ft, House_Price increases by ?)
→ __.....
3. Does the intercept make practical sense? Why or why not? → …….

3.3 Multiple Linear Regression

Multiple regression uses FOUR independent variables simultaneously: Square_Footage, Num_Bedrooms, Num_Bathrooms, and Neighborhood_Quality.

```
model_multiple = LinearRegression()
model_multiple.fit(X_train, y_train)

print('=== Multiple Linear Regression ===')
print(f'Intercept (β0): {model_multiple.intercept_:,.2f}')
print()
for feat, coef in zip(X_train.columns, model_multiple.coef_):
    print(f'  β for {feat:<25}: {coef:>10,.2f}')

# Display as equation
coefs = dict(zip(X_train.columns, model_multiple.coef_))
print()
print('Equation:')
print('House_Price = β0')
for feat, coef in coefs.items():
    print(f'           + {coef:,.2f} × {feat}')
```

Task 3.3 — Interpret Multiple Regression Coefficients
1.For each coefficient, write your interpretation. (Hold all other variables constant.)
Square_Footage coefficient: → ……..

Num_Bedrooms coefficient: → …….

Num_Bathrooms coefficient: → …….

Neighborhood_Quality coefficient:→ …….

2.Which variable has the largest impact on House_Price? Justify your answer.

→ .....

3.4 Polynomial Regression (Degree 2)

Polynomial regression captures non-linear relationships by adding squared (and cross) terms. We still use OLS fitting via LinearRegression, we only transform the features first.

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import Pipeline

# Build a pipeline: polynomial transform → linear regression
poly_pipeline = Pipeline([
    ('poly',  PolynomialFeatures(degree=2, include_bias=False)),
    ('model', LinearRegression())
])

poly_pipeline.fit(X_train, y_train)

# Retrieve the expanded feature names
poly_features = poly_pipeline.named_steps['poly'].get_feature_names_out(X_train.columns)
poly_coefs    = poly_pipeline.named_steps['model'].coef_
poly_intercept = poly_pipeline.named_steps['model'].intercept_

print('=== Polynomial Regression (degree=2) ===')
print(f'Intercept: {poly_intercept:,.2f}')
print(f'Number of features after transformation: {len(poly_features)}')
print()
for name, coef in zip(poly_features, poly_coefs):
    print(f'  {name:<45}: {coef:>12,.4f}')
```

Task 3.4 — Polynomial Features

1. How many features does the dataset have after the degree-2 polynomial transformation?

→ .....

2. Why does the number of features increase? Give the mathematical formula.

→ .....

3. What risk arises from using very high polynomial degrees (e.g., degree=10)?

→ ……___

Part 4 — Model Evaluation

Learning Objectives
• Compute MSE, RMSE, and R^2 for all three models.
• Compare model performance and identify the best model.
• Understand the bias-variance trade-off through evaluation metrics.

4.1 Evaluation Metrics Reference

<table>
<tr><th>Metric</th><th>Formula</th><th>What It Measures</th></tr>
<tr><td>MSE</td><td>mean((y - ŷ) ²)</td><td>Average squared difference between actual and predicted values</td></tr>
<tr><td>RMSE</td><td>√MSE</td><td>Root mean squared error, in the same unit as House_Price</td></tr>
<tr><td>R²</td><td>1 - SS_res/SS_tot
Where:
<table>
<tr><th>Term</th><th>Name</th><th>Formula</th></tr>
<tr><td>SS_{res}</td><td>Residual Sum of Squares</td><td>$\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$</td></tr>
<tr><td>SS_{tot}</td><td>Total Sum of Squares</td><td>$\sum_{i=1}^{n}(y_i - \bar{y})^2$</td></tr>
</table>

y_i= actual value
$\hat{y}_i$= predicted value
$\bar{y}$= mean of actual values
</td><td>Proportion of variance explained by the model (0–1)
<table>
<tr><th>R² Value</th><th>Meaning</th></tr>
<tr><td>R² = 1</td><td>Perfect fit ➔ model explains all variance</td></tr>
<tr><td>R² = 0</td><td>Model explains nothing (no better than predicting the mean)</td></tr>
<tr><td>R² < 0</td><td>Model is worse than just using the mean</td></tr>
<tr><td>R² = 0.85</td><td>Model explains 85% of the variance in the data</td></tr>
</table></td></tr>
</table>

4.2 Compute Metrics for All Three Models

```
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np

# Predictions
y_pred_simple   = model_simple.predict(X_simple_test)
y_pred_multiple = model_multiple.predict(X_test)
y_pred_poly     = poly_pipeline.predict(X_test)

# Metric function
```

```
def evaluate(name, y_true, y_pred):
    mse  = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    r2   = r2_score(y_true, y_pred)
    print(f'--- {name} ---')
    print(f'  MSE  : {mse:>15,.2f}')
    print(f'  RMSE : {rmse:>15,.2f}')
    print(f'  R²   : {r2:>15.4f}')
    print()
    return mse, rmse, r2

r_simple   = evaluate('Simple Linear Regression',   y_test, y_pred_simple)
r_multiple = evaluate('Multiple Linear Regression', y_test, y_pred_multiple)
r_poly     = evaluate('Polynomial Regression (d=2)',y_test, y_pred_poly)
```

4.3 Visualize Predicted vs Actual Values

```
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
configs = [
    ('Simple Regression',     y_pred_simple),
    ('Multiple Regression',   y_pred_multiple),
    ('Polynomial Regression', y_pred_poly),
]

for ax, (title, y_pred) in zip(axes, configs):
    ax.scatter(y_test, y_pred, alpha=0.5, color='#2E5FA3', edgecolors='white', s=40)
    lims = [min(y_test.min(), y_pred.min()), max(y_test.max(), y_pred.max())]
    ax.plot(lims, lims, 'r--', linewidth=2, label='Perfect prediction')
    ax.set_xlabel('Actual House Price (USD)')
    ax.set_ylabel('Predicted House Price (USD)')
    ax.set_title(title, fontweight='bold')
    ax.legend()

plt.suptitle('Predicted vs Actual — All Three Models', fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()
```

4.4 Residual Analysis

Residuals are the differences between actual and predicted values. A good model produces residuals that are randomly distributed around zero.

```
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
```

```
for ax, (title, y_pred) in zip(axes, configs):
    residuals = y_test - y_pred
    ax.scatter(y_pred, residuals, alpha=0.5, color='#E74C3C', edgecolors='white', s=40)
    ax.axhline(0, color='black', linewidth=1.5, linestyle='--')
    ax.set_xlabel('Predicted Values')
    ax.set_ylabel('Residuals')
    ax.set_title(f'Residuals — {title}', fontweight='bold')

plt.tight_layout()
plt.show()
```

4.5 Comparative Results Summary

Model	MSE	RMSE	R^2
Simple Linear Regression	...	...	...
Multiple Linear Regression	...	...	...
Polynomial Regression (d=2)	...	...	...

Task 4 — Model Evaluation Interpretation

Answer the following questions based on your results:

1. Which model achieved the highest R^2 score? What does this mean?

   → …….

2. By how much did adding more features (Multiple vs Simple) reduce the RMSE?

   → …….

3. Did Polynomial Regression improve the results over Multiple Regression? Why or why not?

   → ……

4. Looking at the residual plots, does your best model satisfy the assumptions of linear regression (random, zero-centered residuals)?

   → ……

5. An R^2 of 0.85 means the model explains ……% of the variance in House_Price. Is this a good model for real-estate pricing? Justify.

   → ……..